

SISTEMAS OPERACIONAIS

IV - Threads

Professor: PhD. Eduardo Duarte

SISTEMAS OPERACIONAIS PARTE I

Visão Geral de Threads

Tópicos de Estudo:

- Conceito e Estrutura Básica de Threads
- Execução Multithread e Concorrência
- Compartilhamento de Memória e Recursos do Sistema



O que é uma Thread?

Uma thread é a **unidade básica de utilização da CPU**. Ela é composta por quatro elementos principais:

1. **ID de Thread**
2. **Contador de Programa**
3. **Conjunto de Registradores**
4. **Pilha**

Praticamente todos os sistemas operacionais modernos fornecem recursos que habilitam **um processo a conter múltiplos threads** de controle.

Processo vs. Thread

Processo (Heavyweight)

- **Programa em execução** com espaço de memória próprio e isolado.
- Possui recursos próprios: espaço de endereçamento, arquivos e registradores.
- Troca de contexto é **mais lenta** e dispendiosa para o sistema.
- Oferece **alta independência**: se um processo falha, outros não são afetados diretamente.
- Comunicação entre processos (IPC) exige mecanismos adicionais do SO.

Thread (Lightweight)

- **Unidade básica de CPU** dentro de um processo.
- Compartilha **código, dados e recursos** do processo pai.
- Troca de contexto é **extremamente rápida** e eficiente.
- Cada thread tem seu próprio **ID, Contador, Registradores e Pilha**.
- Se um thread falha criticamente, pode impactar todo o processo.



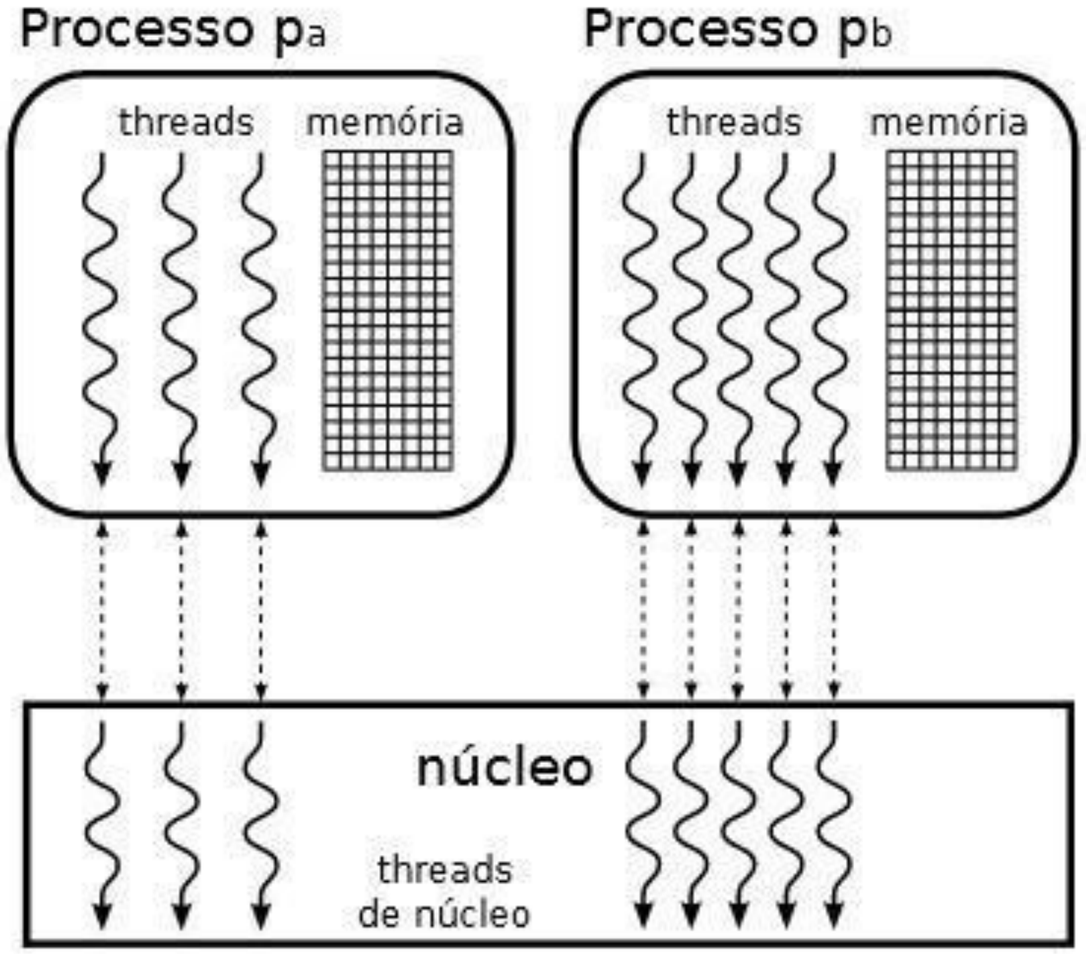
Threads do mesmo processo compartilham:

- **Seção de código**
- **Seção de dados**

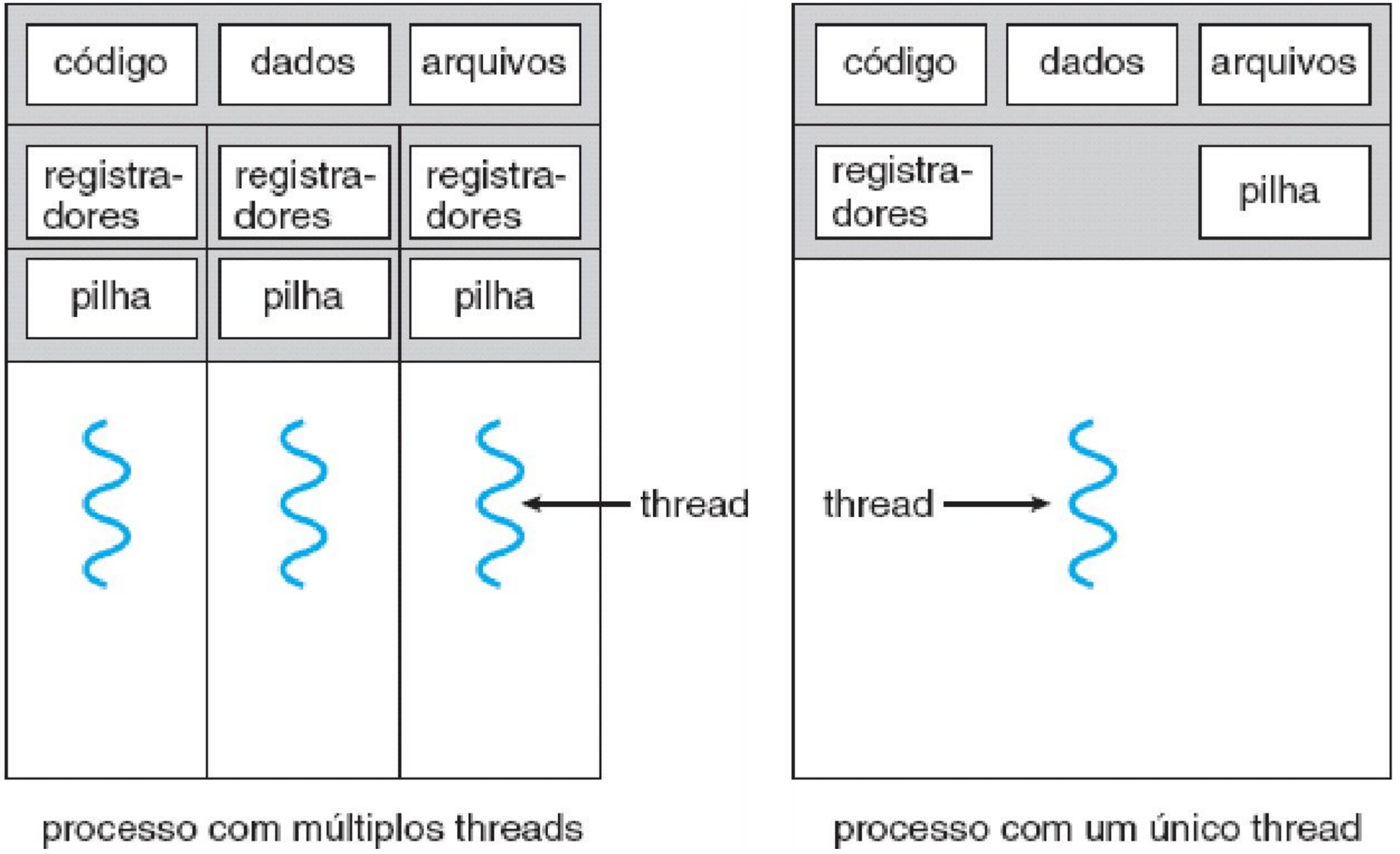
Outros recursos do sistema operacional:

- como arquivos abertos e sinais.

Se um processo tem **múltiplos threads de controle**, ele pode executar **mais de uma tarefa** ao mesmo tempo.







processo com múltiplos threads

processo com um único thread



Exemplos de aplicação

Navegador Web

Múltiplas threads atuando em paralelo:

- Uma thread para **exibir imagens ou texto**.
- Outra thread para **recuperar dados da rede**.

Processador de Textos

Divisão de tarefas entre threads:

- Thread 1: **Exibir elementos gráficos**.
- Thread 2: **Responder ao teclado** do usuário.
- Thread 3: **Verificação ortográfica** em background.

Tentem imaginar como seria se fosse **tudo uma Thread só!**



Exemplos de aplicação

O Cenário: Servidor Web

Demandas de acessos concorrentes:

- Um servidor web ocupado pode ter **diversos clientes**.
- Solicitações ocorrem de forma **concorrente**.

Gargalo: Thread Único

Limitação do modelo tradicional:

- Atendimento de **apenas um cliente por vez**.
- Processo único bloqueia outros acessos.

Uma solução é **tornar o servidor capaz de operar** aceitando múltiplas solicitações!



Exemplos de aplicação

Execução Multitarefa

Em certas situações, uma **única aplicação** pode ser chamada a executar **várias tarefas semelhantes**.

Exemplo: Servidor Web

Um **servidor web** aceita continuamente solicitações de diversos clientes para:

- **Páginas web**
- **Imagens**
- **Arquivos de som**, e assim por diante.



Exemplos de aplicação

Múltiplos Processos

Funcionamento do Modelo:

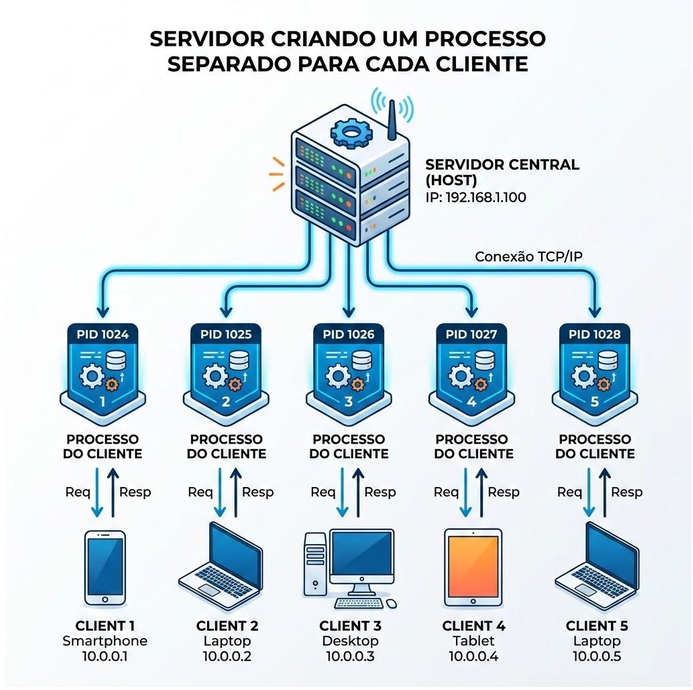
- Quando o servidor receber uma solicitação, ele **criará um processo separado** para atendê-la.

Contexto Histórico

Uso Tradicional:

- Esse método de criação de processos era de **uso comum** antes de os **threads se popularizarem**.

Cada solicitação recebida é tratada por um **processo individual e dedicado** no servidor.



Cada solicitação recebida é tratada por um **processo individual e dedicado** no servidor.



Exemplos de aplicação

Se o novo processo tiver que executar as mesmas tarefas do processo existente, **por que incorrer em todo esse overhead?**



Desvantagens de Múltiplos Processos:

Custo de Criação

Criar um processo é um processo caro, pois exige a alocação de um novo e completo espaço de memória, tabelas de processo e outros recursos do sistema.

Comunicação Lenta

A comunicação entre os processos é mais lenta, já que eles não compartilham a mesma memória e precisam de mecanismos complexos para trocar informações.

Duplicação de Código

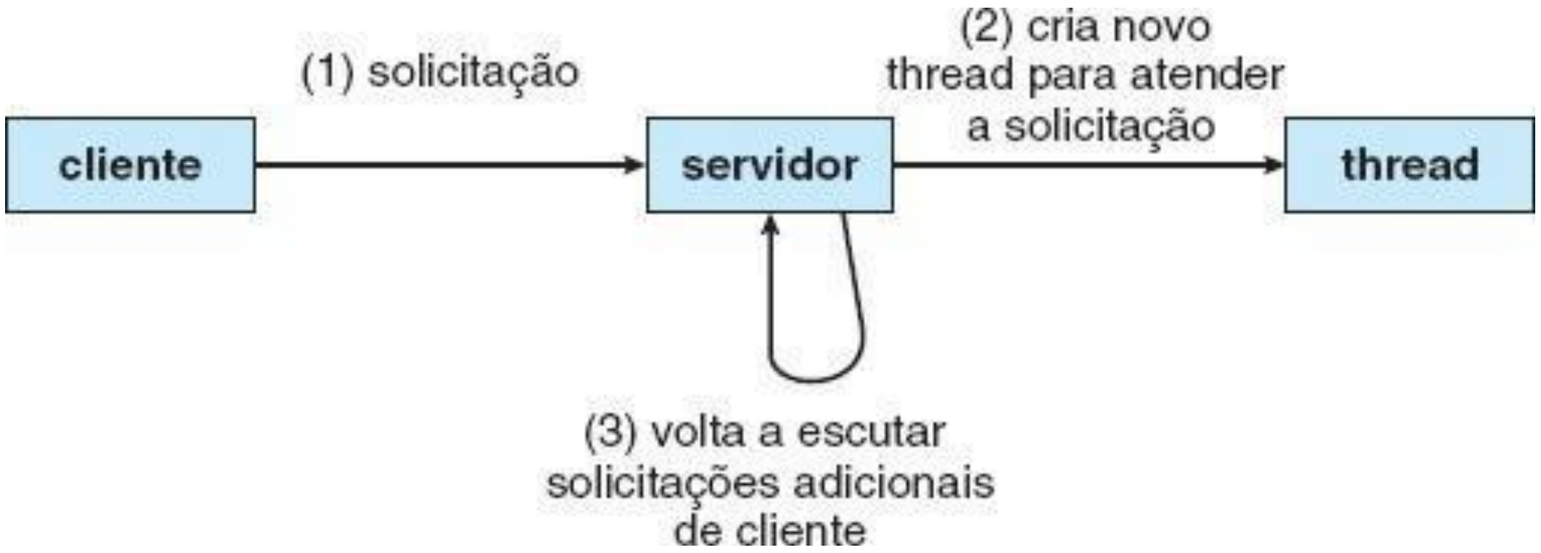
Cada novo processo é uma cópia do programa original, o que leva à duplicação desnecessária de código e dados.



Exemplos de aplicação

Geralmente, é mais eficiente usar um processo contendo **múltiplos threads**.

Se o processo do servidor web for **multithreaded**, o servidor criará um **thread separado** para escutar solicitações de clientes.





Benefícios das Threads: Capacidade de Resposta

Conceito Chave

Tornar uma aplicação interativa multithreaded permite que o programa **continue executando** mesmo que parte dele esteja bloqueada ou realizando uma operação demorada.

Isso aumenta significativamente a **capacidade de resposta** percebida pelo usuário.

Exemplo Prático

Considere um usuário clicando em um botão que executa uma tarefa demorada:

- **Aplicação Single-thread:** Deixaria de responder completamente ao usuário até a conclusão da operação.
- **Aplicação Multithreaded:** Mantém a interface ativa e interativa em background.



Benefícios das Threads: Compartilhamento de Recursos

Processos vs. Threads

Processos: Só podem compartilhar recursos por meio de técnicas explícitas (memória compartilhada ou transmissão de mensagens) organizadas pelo programador.

Threads: Por **default**, compartilham a memória e os recursos do próprio processo ao qual pertencem.

Principal Vantagem

O compartilhamento de código e dados permite que uma aplicação execute **múltiplos threads de atividade** simultaneamente.

Toda essa execução ocorre de forma altamente eficiente dentro do **mesmo espaço de endereçamento**.



Benefícios das Threads: Economia

Economia de Recursos

A alocação de memória e recursos para a criação de processos é **dispendiosa**.

Como as threads **compartilham os recursos** do processo ao qual pertencem, é muito mais econômico criá-las e alternar seus contextos.

Exemplo Prático (Solaris)

Criar e gerenciar processos exige um overhead significativamente maior do que threads:

- **Criação de Processo: cerca de 30x mais lenta do que a de uma thread.**
- **Troca de Contexto: cerca de 5x mais lenta em comparação a threads.**



Benefícios das Threads: Escalabilidade

Arquitetura Multiprocessadora

Execução em Paralelo: Os benefícios da criação de múltiplos threads podem ser ainda maiores em uma arquitetura multiprocessadora.

Os threads podem ser executados simultaneamente em **diferentes núcleos de processamento**.

Processos Single-Threaded

Limitação de Hardware: Um processo com um único thread só pode ser executado em **um único processador**, independentemente de quantos estiverem disponíveis.

Exploramos mais essa questão na próxima seção.

PARTE II

Programação Multicore



Evolução das CPUs

Sistemas Múltiplas CPUs

Nos primórdios dos projetos de computadores, os sistemas dependiam exclusivamente de uma **única CPU**.

Para atender à crescente demanda por desempenho computacional, a arquitetura evoluiu para a integração de **várias CPUs** independentes.

Sistemas Multicore

A evolução mais recente no projeto de sistemas é a inclusão de **múltiplos núcleos** de processamento integrados em um **único chip**.

Essa abordagem é conhecida como tecnologia **multicore** ou multiprocessadores integrados.



Evolução das CPUs

A programação com **múltiplos threads** fornece um mecanismo para o uso mais eficiente dos **múltiplos núcleos** de computação e o aumento da concorrência.

Exemplo: Considere uma aplicação com quatro threads.

Sistema com Único Núcleo de Computação

Em um sistema com um **único núcleo**, a **concorrência** significa simplesmente que a execução dos threads será **intercalada por meio do tempo**.

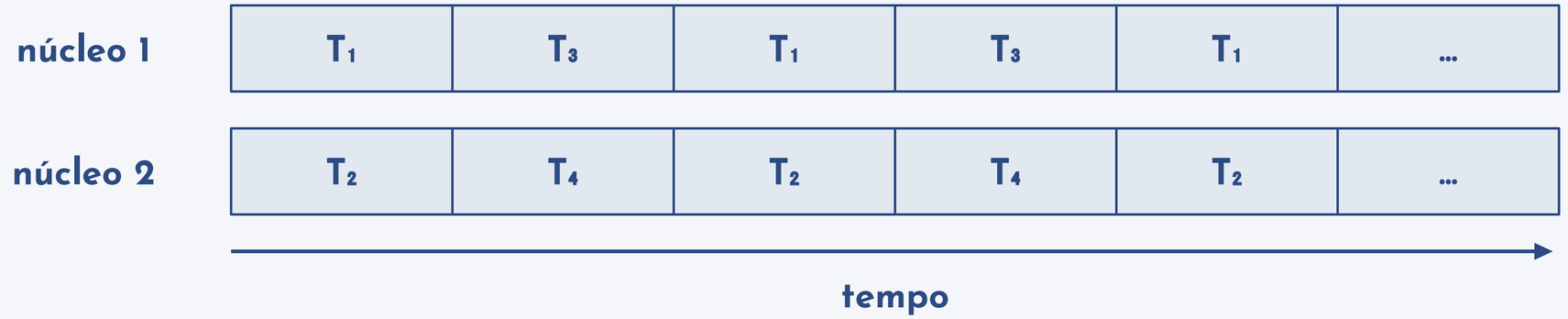




Evolução das CPUs

Em um sistema com **múltiplos núcleos**, no entanto, **concorrência** significa que os threads podem ser **executados em paralelo**, já que o sistema pode atribuir um thread separado a cada núcleo.

Execução Paralela em Sistema Multicore





Diferença entre Paralelismo e Concorrência

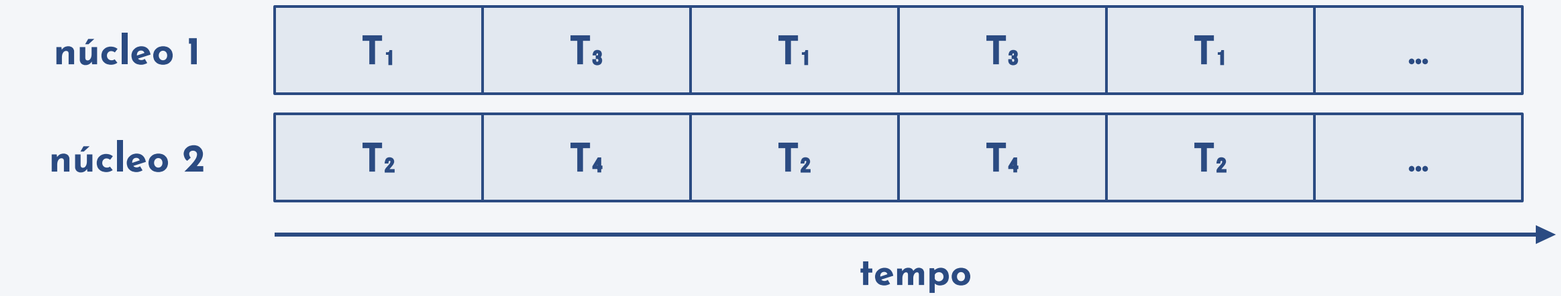
Concorrência: Execução Intercalada (Núcleo Único)

Múltiplas tarefas progridem intercalando a execução no tempo.



Paralelismo: Execução Simultânea (Múltiplos Núcleos)

Múltiplas tarefas são executadas ao mesmo tempo em núcleos separados.





Diferença entre Paralelismo e Concorrência

Sistema Paralelo

Um sistema é **paralelo** quando pode executar mais de uma tarefa **simultaneamente**.

Sistema Concorrente

Um sistema é **concorrente** quando dá suporte a mais de uma tarefa, permitindo que todas elas **progridam**.

Conclusão: É possível haver concorrência sem paralelismo!



Diferença entre Paralelismo e Concorrência

O Contexto do Núcleo Único

Quando os computadores tinham um **único processador** (ou apenas um núcleo), os **schedulers da CPU** eram projetados para dar a impressão de paralelismo, **permutando rapidamente os processos** no sistema e permitindo que cada um progredisse.

Esses processos eram executados concorrentemente, mas não em paralelo.



Desafios da Programação

Projetistas de Sistemas Operacionais

Devem escrever algoritmos de **scheduling** que utilizem múltiplos núcleos de processamento para permitir a **execução paralela**.

Programadores de Aplicações

O desafio é a **modificação dos programas** existentes e o projeto de novos programas preparados para **múltiplos threads**.



Desafios da Programação

1. Identificação de Tarefas

Envolve a **análise detalhada das aplicações** em busca de áreas e rotinas que possam ser divididas em tarefas **separadas e concorrentes**.

O ideal é que as tarefas sejam independentes para serem executadas em paralelo em núcleos individuais.

Desafio Prático: Identificação de Tarefas

Como dividir um **Contador de Palavras** em tarefas concorrentes?

Pense: Como garantir que a divisão do texto resulte em tarefas balanceadas e independentes?

Desafio Prático: Identificação de Tarefas

Como dividir um **Contador de Palavras** em tarefas concorrentes?

1. Leitura & Divisão

Ler o texto de entrada e dividi-lo em blocos/segmentos menores para processamento.

Pense: Como garantir que a divisão do texto resulte em tarefas balanceadas e independentes?

Desafio Prático: Identificação de Tarefas

Como dividir um **Contador de Palavras** em tarefas concorrentes?

1. Leitura & Divisão

Ler o texto de entrada e dividi-lo em blocos/segmentos menores para processamento.

2. Contagem Parcial

Contar a frequência das palavras em cada bloco de forma independente e paralela.

Pense: Como garantir que a divisão do texto resulte em tarefas balanceadas e independentes?

Desafio Prático: Identificação de Tarefas

Como dividir um **Contador de Palavras** em tarefas concorrentes?

1. Leitura & Divisão

Ler o texto de entrada e dividi-lo em blocos/segmentos menores para processamento.

2. Contagem Parcial

Contar a frequência das palavras em cada bloco de forma independente e paralela.

3. Consolidação

Combinar os resultados parciais de todos os núcleos para gerar o total final.

Pense: Como garantir que a divisão do texto resulte em tarefas balanceadas e independentes?



Desafios da Programação: Equilíbrio

1. Distribuição de Esforço

Além de identificar tarefas que possam ser executadas em paralelo, os programadores também devem assegurar que elas desenvolvam **esforço igual e de mesmo valor**.

2. Análise Custo-Benefício

Em alguns casos, determinada tarefa pode não contribuir tanto para o processo em geral. O uso de um **núcleo de execução separado** para realizá-la pode não valer a pena.

Pense: Como garantir que a carga de trabalho seja balanceada para evitar núcleos ociosos ou sobrecarregados?



Desafios da Programação: Dados e Testes

1. Divisão de Dados

Assim como as aplicações, os dados manipulados devem ser **divididos para execução** em núcleos separados.

2. Teste e Depuração

Com múltiplos caminhos concorrentes, testar e depurar é **muito mais complexo** do que em aplicação com único thread.

Pense: Como dividir os dados mantendo a corretude e facilidade de teste das tarefas?



Desafios da Programação: Dependência e Sincronização

1. Dependência de Dados

Os dados acessados pelas tarefas devem ser examinados para **determinar dependências** entre duas ou mais tarefas.

2. Papel da Sincronização

Quando uma tarefa depende de outra, a execução deve ser **sincronizada** para assegurar a ordem correta e a integridade dos dados.

Pense: Como garantir a sincronização adequada sem gerar Gargalos de Desempenho?

Problemas da Falta de Sincronização e Gargalos

1. Riscos sem Sincronização

• **Condição de Corrida (Race Condition):** Ocorre quando múltiplas tarefas acessam e modificam os mesmos dados simultaneamente sem controle.

• **Inconsistência de Dados:** O resultado final torna-se imprevisível e incorreto.

• **Exemplo Prático:** Duas threads tentando atualizar o saldo de uma mesma conta bancária ao mesmo tempo. Sem trava, uma alteração pode sobrescrever a outra, gerando saldo incorreto.

2. Por que gera Gargalo?

• **Espera Excessiva (Contenção):** Para evitar a corrupção de dados, usam-se mecanismos de trava (locks).

• **Serialização Forçada:** Se muitas threads disputam a mesma trava, elas precisam esperar em fila, bloqueando a execução.

• **Perda de Paralelismo:** Processadores de múltiplos núcleos ficam ociosos esperando pela liberação dos dados, reduzindo o desempenho geral do sistema.

Conclusão: O Desafio do Balanço

A sincronização é essencial para a **corretude dos dados**, mas seu excesso transforma o processamento paralelo em **execução serial**, limitando o ganho de desempenho.



Lei de Amdahl

1. O Conceito

É uma fórmula que **identifica ganhos de desempenho** potenciais ao adicionar núcleos de computação em uma aplicação.

- Considera componentes **seriais** (não paralelos) e **paralelos**.

2. Variáveis da Fórmula

- **S:** Parte da aplicação executada **serialmente**.
- **N:** Número de **núcleos de processamento**.
- **(1 - S):** Fração da aplicação que pode ser **paralelizada**.

$$aceleração \leq \frac{1}{S + \frac{(1-s)}{N}}$$



Lei de Amdahl: Exemplo Prático

1. Cenário Inicial (2 Núcleos)

- **Composição:** Aplicação **75% paralela** e **25% serial**.
- **Infraestrutura:** Sistema operando com **2 núcleos** de processamento.
- **Resultado:** Obtém-se uma aceleração de **1,6 vez**.

2. Expansão (4 Núcleos)

- **Adição de Recursos:** Adicionando mais 2 núcleos (**total de 4 núcleos**).
- **Novo Resultado:** A aceleração passa para **2,28 vezes**.
- **Comportamento:** O ganho de velocidade não dobra devido ao gargalo dos **25% seriais**.

Conclusão do Exemplo

Dobrar os núcleos (de 2 para 4) eleva a aceleração de **1,6x para 2,28x**, exemplificando como a fração serial limita o ganho escalar absoluto.



Lei de Amdahl: Limite Teórico

1. Comportamento Assintótico

- **Convergência:** Conforme N (núcleos) tende ao infinito, o ganho de velocidade atinge um teto.
- **Fórmula do Limite:** A aceleração converge para **1 / S** .

2. Exemplo Prático (S = 40%)

- **Fração Serial (S):** 40% da aplicação (0,4) executa obrigatoriamente de forma serial.
- **Cálculo do Máximo:** 1 / 0,4 = **2,5 vezes** .

Conclusão do Limite Teórico

Mesmo adicionando **infinitos núcleos** , a aceleração não passará de **2,5x** devido ao gargalo dos 40% seriais.



Tipos de Paralelismo

Em geral, a computação paralela divide-se em duas abordagens principais:

1. Paralelismo de Dados

- **Foco Principal:** Distribuição de subconjuntos dos mesmos dados por múltiplos núcleos de computação.
- **Execução:** Cada núcleo executa exatamente a mesma operação em sua respectiva porção de dados.
- **Aplicação Típica:** Processamento de matrizes, vetores e operações em lote sobre grandes volumes de dados.

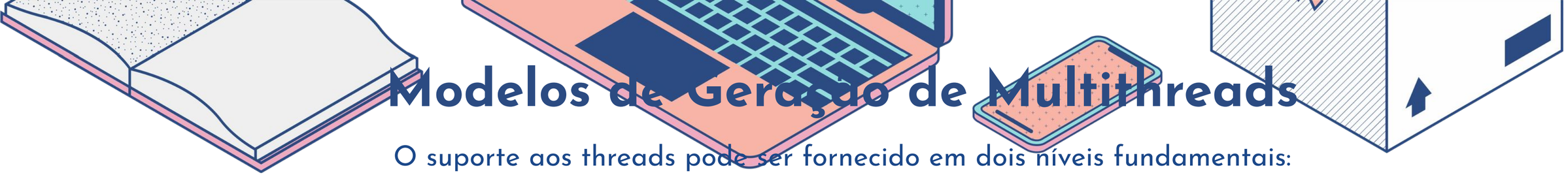
2. Paralelismo de Tarefas

- **Foco Principal:** Distribuição de tarefas distintas (threads) em vários núcleos de computação separados.
- **Execução:** Cada thread realiza uma única operação exclusiva de forma independente.
- **Flexibilidade de Dados:** Diferentes threads podem operar tanto sobre os mesmos dados quanto sobre dados distintos.

PARTE III

Modelos de Geração de Multithreads

Visão geral e arquitetura do suporte a threads em nível de usuário e de kernel



Modelos de Geração de Multithreads

O suporte aos threads pode ser fornecido em dois níveis fundamentais:

1. Threads de Usuário

- **Suporte:** Suportados acima do nível do kernel.
- **Gerenciamento:** Gerenciados sem o suporte do kernel.
- **Execução:** Controlados no espaço do usuário por uma biblioteca de threads.

2. Threads de Kernel

- **Suporte:** Suportados diretamente pelo próprio kernel.
- **Gerenciamento:** Gerenciados pelo sistema operacional.
- **Execução:** Escalonamento e operações realizados pelo núcleo do SO.



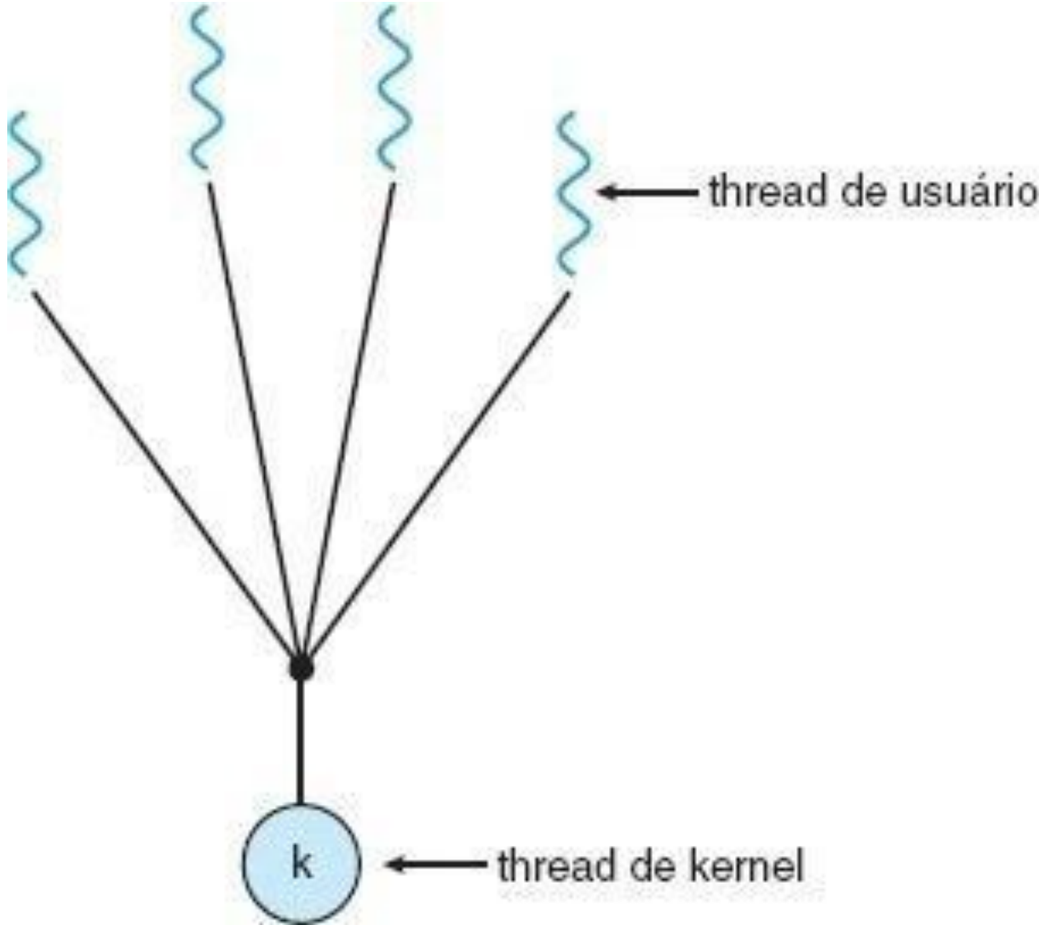
Modelo Muitos-para-Um

📌 Funcionamento & Eficiência

- Mapeia **muitos threads de usuário** para apenas **um thread de kernel**.
- O gerenciamento ocorre totalmente no espaço do usuário via **biblioteca de threads**, tornando a criação e troca de contexto **altamente eficiente**.

⚠️ Limitação Principal

Se um único thread realizar uma **chamada de sistema bloqueadora**, o **processo inteiro será bloqueado**, impedindo a execução dos demais threads.





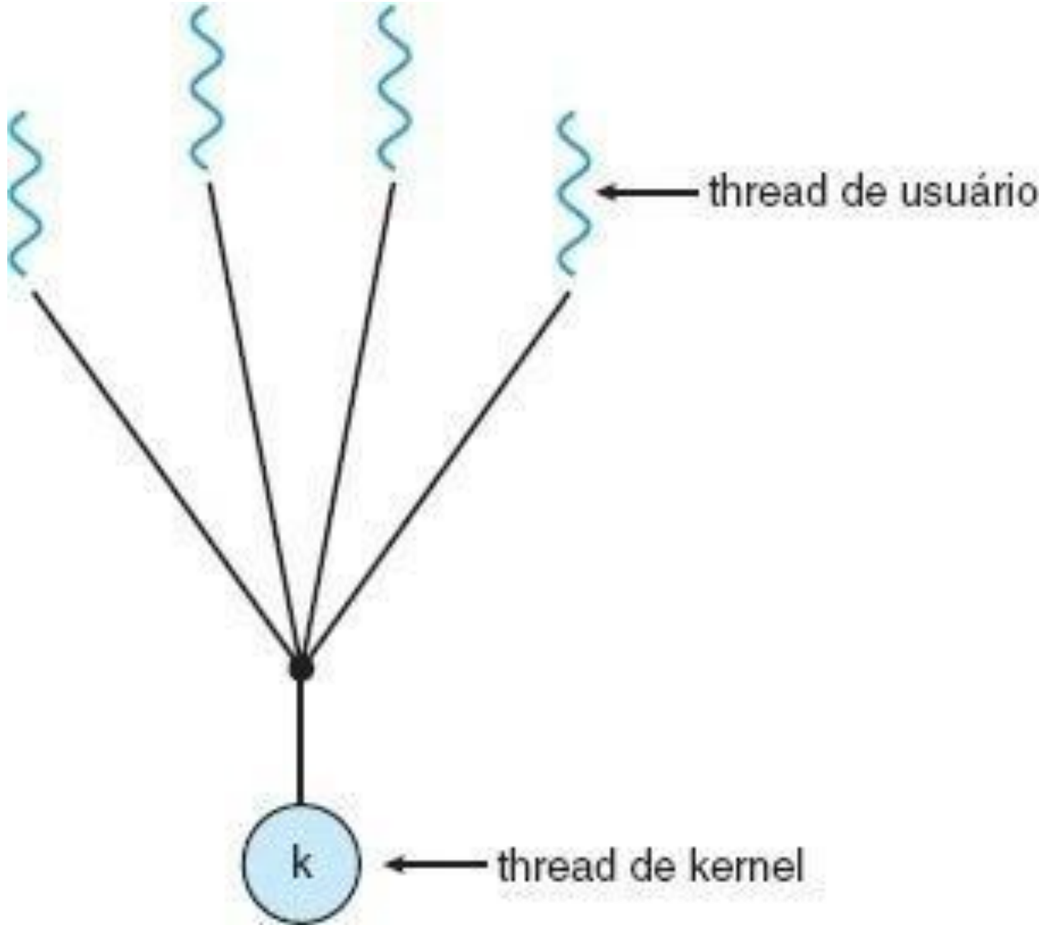
Modelo Muitos-para-Um

⚡ Gargalo em Multicore

- Apenas **um thread por vez pode acessar o kernel**, tornando muitos threads incapazes de executar em paralelo em sistemas multicore.

📉 Declínio de Uso

- Poucos sistemas continuam a usar este modelo devido à sua **incapacidade de aproveitar múltiplos núcleos** de processamento.





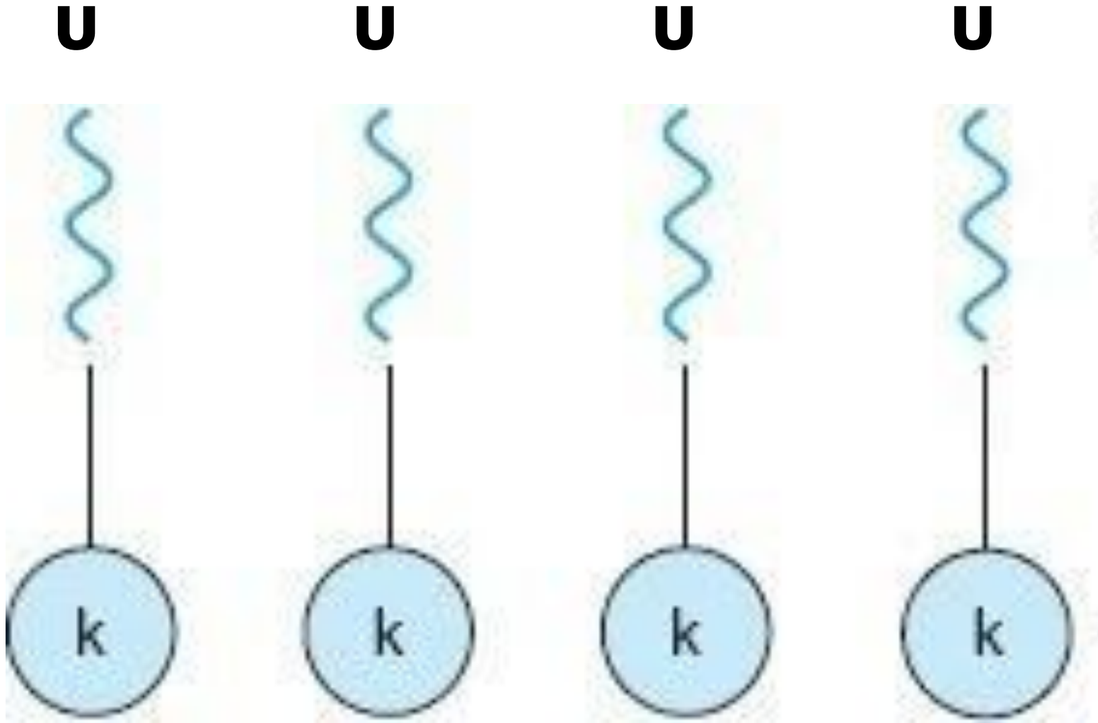
Modelo Um-Para-Um

📌 Funcionamento & Mapeamento

- Mapeia cada **thread de usuário para um thread de kernel**.

⚡ Ganho em Concorrência

- Fornece mais concorrência do que o modelo muitos-para-um.
- Permite que outro thread seja **executado normalmente** quando um thread faz uma **chamada de sistema bloqueadora**.





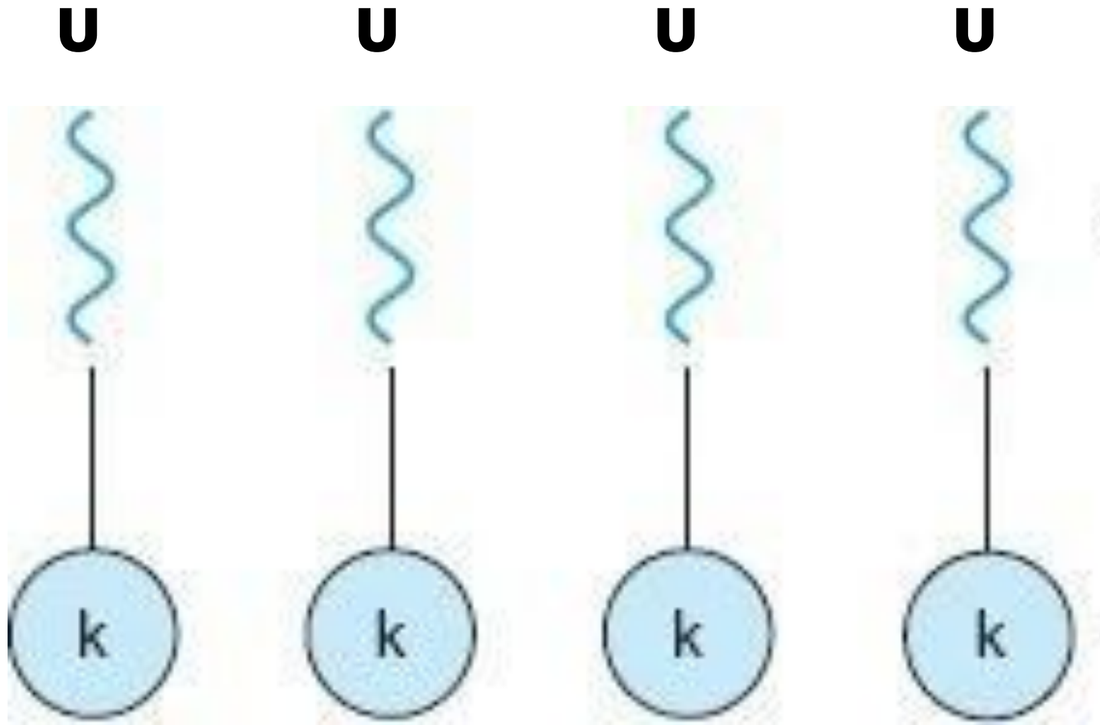
Modelo Um-Para-Um

⚡ Execução em Paralelo

- Permite que **múltiplos threads** sejam executados em paralelo em multiprocessadores.

⚠️ Principal Desvantagem

- A criação de um thread de usuário **requer a criação do thread de kernel** correspondente.
- Já que o overhead de criação de threads de kernel pode sobrecarregar o desempenho de uma aplicação, a maioria das implementações desse modelo **restringe o número de threads**





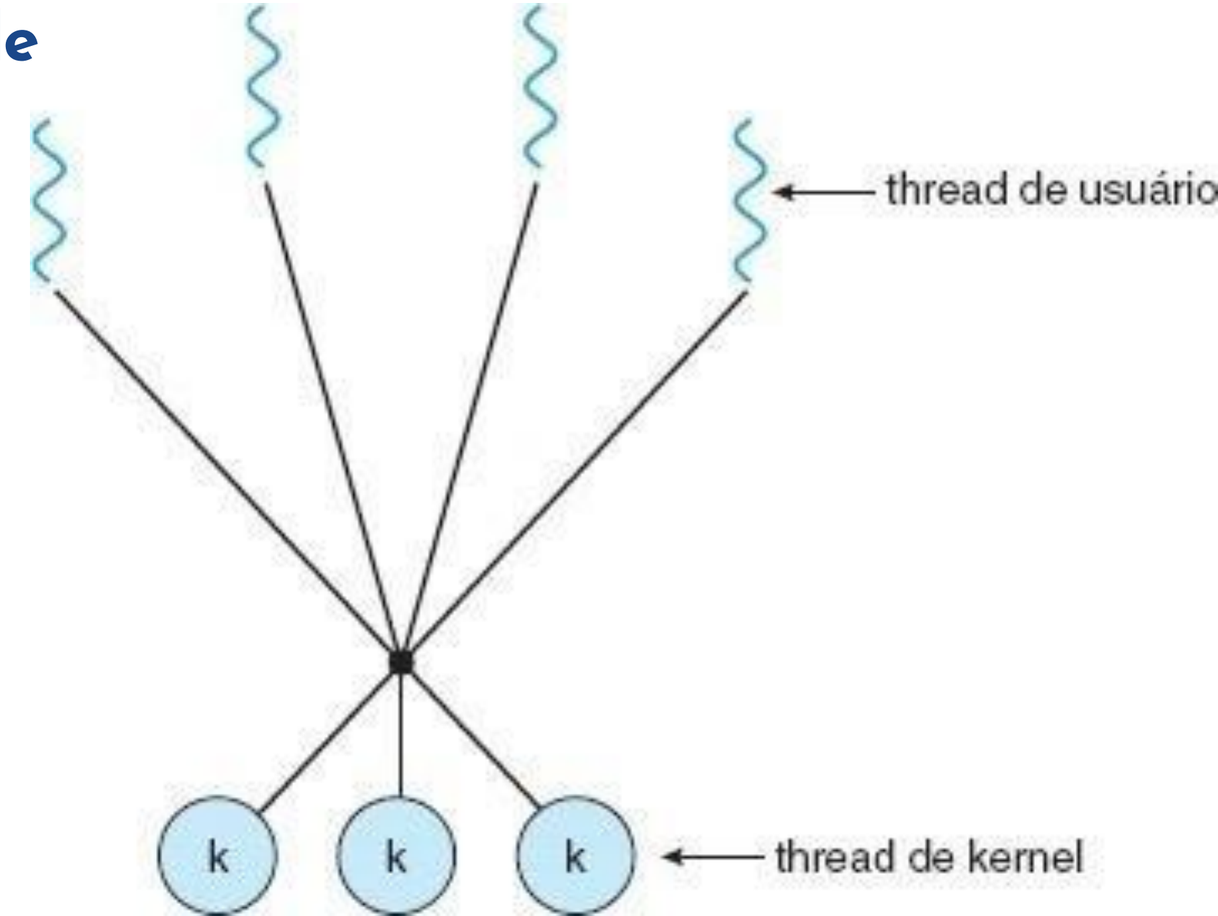
Modelo Muitos-Para-Muitos

📌 Funcionamento & Mapeamento

- Multiplexa **muitos threads de nível de usuário** para um número **menor ou igual de threads de kernel**.

⚙️ Dimensionamento Flexível

- O número de threads de kernel pode ser **específico para determinada aplicação ou máquina**.

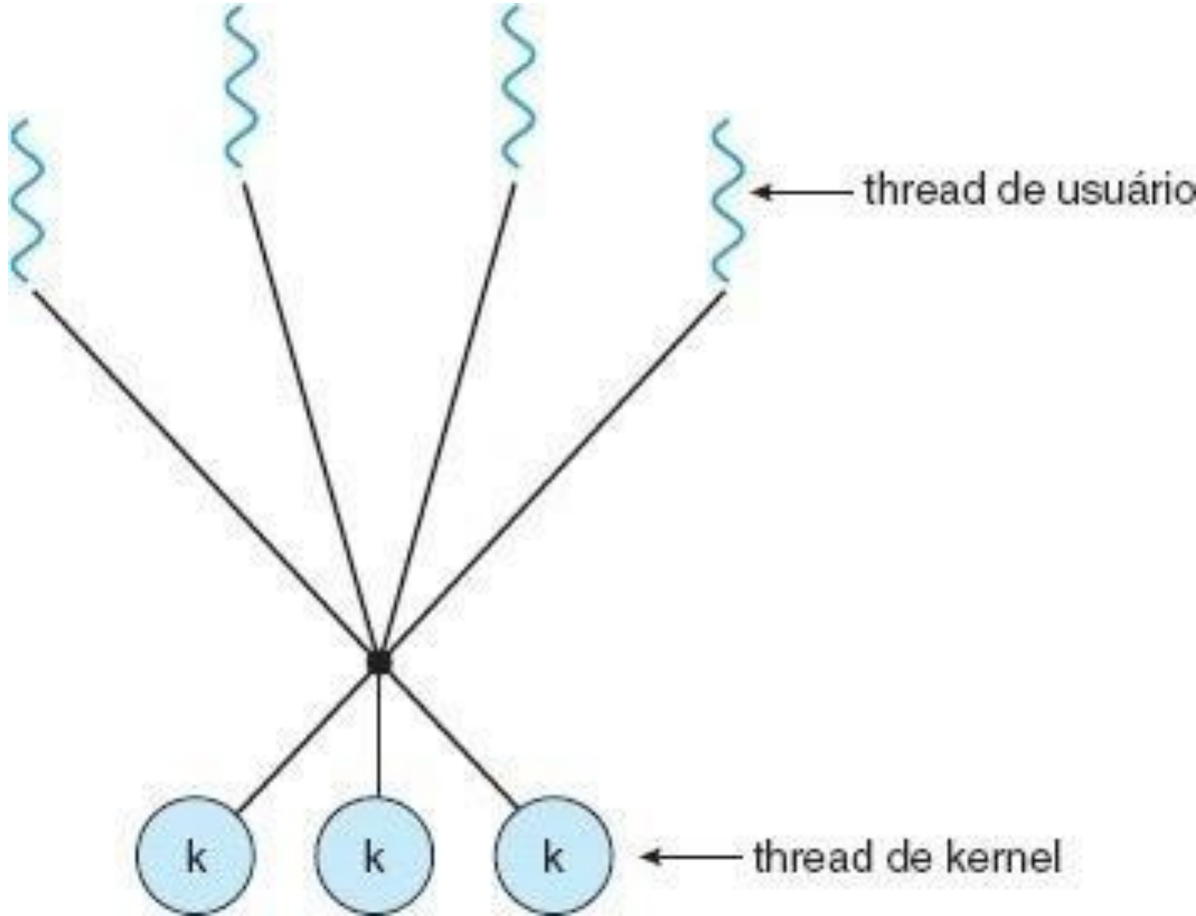




Modelo Muitos-Para-Muitos

⚡ Vantagens & Paralelismo

- Desenvolvedores podem criar **quantos threads de usuário forem necessários**.
- Threads de kernel podem ser **executados em paralelo** em um multiprocessador.



🔄 Tratamento de Bloqueios

- Ao executar uma chamada de sistema bloqueadora, o kernel pode designar **outro thread para execução**.



Modelo Muitos-Para-Muitos

🔀 Modelo de Dois Níveis

- Multiplexa **muitos threads de usuário** para um número **menor ou igual de threads de kernel**.
- Permite também que um thread de usuário seja **limitado a um thread de kernel** específico.

⚡ Principal Vantagem

- Combina a **flexibilidade do modelo muitos-para-muitos** com a **garantia de execução/recursos** para threads críticos sem sofrer bloqueios.

💡 Denominação

- Essa variação é conhecida como **modelo de dois níveis** (Two-Level Model).

